Project Title:

Smart Hospital Queue Management System

Submitted by:

Name: Rohit Raj

Registration no.: 12406063

Name: Amrit Tr

Registration no.: 12403676

Name: Vyom Varun

Registration no.: 12406065

Under the Guidance of:

Mahipal Singh Papola Sir & Deepak Sir

Course:

DSA Bootcamp for Industry

Abstract

Project Overview:

Hospitals frequently experience overcrowding and long patient waiting times due to the traditional first-come, first-served (FCFS) queue management approach. This method often fails to prioritize patients based on the severity of their medical condition, resulting in delayed treatment for emergency cases and inefficient utilization of hospital resources. Managing patient flow efficiently while ensuring fairness and timely medical attention remains a significant challenge in modern healthcare systems.

Problem Statement:

The **Smart Hospital Queue Management System (MediQueue AI)** is a web-based application designed to address these challenges by implementing a priority-driven patient management system using advanced **Data Structures and Algorithms (DSA)**. The system intelligently determines the order in which patients are served by considering factors such as emergency level, age, waiting time, appointment status, and doctor availability. Unlike conventional queue systems, the proposed solution ensures that patients requiring immediate medical attention are prioritized without compromising the overall efficiency of hospital operations.

Proposed Solution:

The application provides dedicated modules for patients, doctors, receptionists, and administrators. Patients can register, book appointments, and monitor their queue status in real time. Receptionists are responsible for patient registration and priority assignment, while doctors receive an automatically organized patient list based on priority. Administrators can monitor hospital activities, manage users, and analyze queue statistics through an interactive dashboard. This integrated workflow improves communication between different hospital departments and enhances patient service quality.

Technologies Used:

The project is developed using modern web technologies. The frontend is built with **React.js**, **TypeScript**, **HTML5**, **CSS3**, and **Tailwind CSS** to provide a responsive and user-friendly interface. **Firebase Authentication** is used for secure user login, while **Firestore Database** stores patient records, appointments, and queue information in real time. Additional tools such as **Vite**, **Git**, and **GitHub** are used for application development, version control, and deployment.

DSA Concepts Used:

A key highlight of the project is the implementation of multiple Data Structures and Algorithms to solve real-world healthcare problems efficiently. A **Priority Queue** implemented using a **Binary Max Heap** is used to schedule patients according to their calculated priority score. A **Queue** manages the normal patient flow, while a **HashMap** enables fast retrieval of patient and doctor information. A **Graph** represents relationships between doctors, departments, and patients, allowing efficient resource organization. Searching and sorting techniques are also incorporated to improve system performance and data management.

Expected Outcomes:

The proposed system aims to reduce patient waiting time, improve emergency response, optimize doctor utilization, and provide a fair and efficient queue management process. By integrating advanced DSA concepts with modern web technologies, the project demonstrates the practical application of algorithms in healthcare management while offering a scalable, reliable, and user-friendly solution for hospitals and clinics. The successful implementation of this system highlights how data structures can significantly enhance operational efficiency and patient satisfaction in real-world medical environments.

Introduction

Background

Efficient patient queue management is one of the most important aspects of healthcare administration. Every day, hospitals and clinics handle a large number of patients with varying medical conditions. Traditional queue management systems generally follow the First-Come-First-Served (FCFS) approach, where patients are treated in the order they arrive. Although this method is simple to implement, it often fails to address emergency situations where critically ill patients require immediate medical attention. As a result, patients with severe conditions may experience unnecessary delays, affecting both treatment quality and overall hospital efficiency.

With the rapid advancement of information technology, hospitals are increasingly adopting digital healthcare solutions to automate administrative tasks and improve patient care. Smart queue management systems help healthcare providers organize patient flow, reduce waiting times, optimize doctor utilization, and improve operational efficiency. By incorporating Data Structures and Algorithms (DSA), these systems can process patient information intelligently and provide priority-based scheduling rather than relying solely on arrival time.

The Smart Hospital Queue Management System (MediQueue AI) is designed to solve these challenges by implementing efficient DSA techniques such as Priority Queue, Binary Heap, Queue, HashMap, and Graph. The project demonstrates how these algorithms can be applied in a real-world healthcare environment to create an intelligent and scalable patient management system.

Problem Statement

Many hospitals still rely on manual or First-Come-First-Served (FCFS) queue management systems. These approaches do not consider the urgency of a patient's medical condition, leading to long waiting times for emergency cases and inefficient utilization of hospital resources. During peak hours, hospital staff often find it difficult to manage patient flow efficiently, resulting in overcrowding, delayed consultations, and reduced patient satisfaction.

Another challenge is the lack of real-time queue monitoring and effective coordination between patients, doctors, and hospital administrators. Patients are unable to accurately estimate their waiting time, while doctors have limited visibility into the

overall patient queue. These limitations reduce operational efficiency and negatively impact the quality of healthcare services.

Proposed Solution

The proposed Smart Hospital Queue Management System introduces a priority-based scheduling mechanism that serves patients according to their medical urgency rather than their arrival time. Each patient is assigned a priority score based on factors such as emergency level, age, waiting time, appointment status, and doctor availability.

The system utilizes a Priority Queue implemented using a Binary Max Heap to automatically arrange patients according to their calculated priority. Patients with higher priority are treated first, ensuring that emergency cases receive immediate medical attention while maintaining fairness for other patients.

The application also provides separate dashboards for patients, doctors, receptionists, and administrators. Patients can register, book appointments, and monitor their queue status. Receptionists manage registrations and assign priorities, doctors receive an automatically organized patient list, and administrators monitor hospital operations through real-time dashboards and analytics.

Objectives of the Project

The primary objectives of the Smart Hospital Queue Management System are:

- To develop a smart patient queue management system using Data Structures and Algorithms.
- To reduce patient waiting time through intelligent priority-based scheduling.
- To ensure that emergency patients receive immediate medical attention.
- To improve hospital workflow by automating queue management.
- To demonstrate the practical implementation of Priority Queue, Heap, Queue, HashMap, and Graph in a real-world application.
- To provide an efficient, user-friendly, and scalable healthcare management solution.

3.5 Scope of the Project

The Smart Hospital Queue Management System is designed for hospitals, clinics, and healthcare centers that require efficient patient scheduling and queue management. The system supports multiple users, including patients, doctors, receptionists, and administrators, each with dedicated functionalities.

The project focuses on patient registration, appointment scheduling, priority calculation, doctor assignment, live queue monitoring, and patient record management. It can be further enhanced with AI-based disease prediction, QR code check-in, SMS notifications, online consultation, mobile application support, and integration with electronic health record (EHR) systems.

3.6 Technologies Used

The project is developed using modern web technologies that ensure high performance, scalability, and ease of maintenance.

Frontend

- React.js
- TypeScript
- HTML5
- Tailwind CSS
- Vite

Backend & Database

- Firebase Authentication
- Firebase Firestore

Development Tools

- Git
- GitHub
- Visual Studio Code

System Modules

The application consists of several functional modules that work together to provide a complete hospital queue management solution.

Patient Module

- Patient Registration
- Appointment Booking
- Live Queue Status
- Estimated Waiting Time

Reception Module

- Patient Registration
- Priority Assignment
- Queue Management

Doctor Module

- View Priority Queue
- Treat Next Patient
- Update Consultation Status

Admin Module

- User Management
- Queue Analytics
- Hospital Monitoring
- System Configuration

3.8 Significance of Data Structures and Algorithms

Data Structures and Algorithms play a crucial role in improving the efficiency of the proposed system.

- **Priority Queue** is used to arrange patients according to their priority score.
- **Binary Max Heap** enables efficient insertion and deletion operations while maintaining queue order.
- **Queue** manages normal patient flow where priority is equal.
- **HashMap** provides fast access to patient and doctor records.
- **Graph** represents relationships between doctors, departments, and patients.
- **Searching and Sorting Algorithms** improve data retrieval and record organization.

The integration of these DSA concepts demonstrates how algorithmic techniques can solve practical healthcare management problems while improving system performance and reducing computational complexity.

Algorithm

Overview

The Smart Hospital Queue Management System uses several Data Structures and Algorithms (DSA) to manage patient queues efficiently and improve hospital workflow. Unlike traditional First-Come-First-Served (FCFS) systems, the proposed solution prioritizes patients based on emergency level, age, appointment status, and waiting time. The core objective of the algorithmic design is to ensure that critical patients receive immediate medical attention while maintaining fairness and efficiency for all users.

The system integrates multiple data structures, including Priority Queue, Binary Max Heap, Queue, HashMap, and Graph. Each data structure is selected based on its efficiency and suitability for solving a specific problem within the application.

Priority Queue Algorithm

Purpose

The Priority Queue is the primary data structure used in the system. It stores patients according to their calculated priority score rather than their arrival time. Patients with higher priority are always treated before patients with lower priority.

Working

1. Register patient details.
2. Calculate the priority score.
3. Insert the patient into the Priority Queue.
4. Maintain queue order using a Binary Max Heap.
5. Serve the patient with the highest priority.
6. Remove the patient after consultation.

Time Complexity

- Insert: **O(log n)**
- Delete Highest Priority: **O(log n)**
- Peek Highest Priority: **O(1)**

Advantages

- Efficient scheduling
- Faster emergency response
- Reduced waiting time
- Better hospital resource utilization

Binary Max Heap

Purpose

The Priority Queue is implemented using a Binary Max Heap. The heap ensures that the patient with the highest priority remains at the root of the tree, enabling quick retrieval and removal.

Working

- Insert new patient.
- Compare with parent node.
- Swap if priority is higher.
- Continue until heap property is restored.
- During deletion, replace the root with the last element and perform heapify.

Time Complexity

- Insert: **O(log n)**
- Delete: **O(log n)**
- Heapify: **O(log n)**

Advantages

- Efficient priority management
- Faster insertion and deletion
- Scalable for large patient queues

Queue Algorithm

Purpose

A Queue is used to manage patients with equal priority and maintain orderly processing where required.

Working

1. Add patient at the rear.
2. Remove patient from the front.
3. Continue until the queue becomes empty.

Time Complexity

- Enqueue: **O(1)**
- Dequeue: **O(1)**
- Front: **O(1)**

Advantages

- Fair processing order
- Simple implementation
- Efficient patient flow

4.5 HashMap Algorithm

Purpose

HashMap stores patient and doctor information using unique IDs as keys. It enables quick retrieval, updating, and deletion of records.

Working

1. Generate unique key.
2. Store patient object.
3. Search using key.
4. Update or delete records when required.

Time Complexity

- Insert: **O(1)**
- Search: **O(1)**
- Delete: **O(1)**

Advantages

- Fast lookup
- Efficient storage
- Reduces search time

Graph Algorithm

Purpose

The Graph data structure represents relationships between hospital departments, doctors, and patients. It helps organize hospital resources logically.

Working

- Each department and doctor is represented as a node.
- Connections between them are represented as edges.
- Relationships can be traversed whenever required.

Time Complexity

- Add Vertex: **O(1)**
- Add Edge: **O(1)**
- BFS/DFS Traversal: **O(V + E)**

Advantages

- Efficient relationship management
- Easy resource visualization
- Scalable hospital structure

Patient Priority Calculation Algorithm

Purpose

The system calculates a priority score for every patient before adding them to the queue.

Factors Considered

- Emergency Level
- Patient Age
- Waiting Time
- Appointment Status

Steps

1. Collect patient information.
2. Assign weights for each factor.
3. Calculate total priority score.
4. Insert patient into Priority Queue.
5. Highest score receives highest priority.

Advantages

- Fair scheduling
- Emergency-first treatment
- Dynamic queue management

Waiting Time Estimation Algorithm

Purpose

Estimate the expected waiting time for each patient.

Steps

1. Determine patient's position in the queue.
2. Calculate the number of patients ahead.
3. Estimate average consultation time.

4. Display approximate waiting time.

Advantages

- Better patient experience
- Reduced uncertainty
- Improved hospital planning

Overall Algorithm Workflow

1. Patient Registration
2. Priority Score Calculation
3. Store Patient Record
4. Insert into Priority Queue
5. Doctor Views Highest Priority Patient
6. Patient Consultation
7. Remove Patient from Queue
8. Update Database
9. Display Updated Queue

Summary

The Smart Hospital Queue Management System combines multiple Data Structures and Algorithms to provide an efficient, scalable, and intelligent queue management solution. The Priority Queue and Binary Max Heap serve as the core scheduling mechanism, while Queue, HashMap, and Graph support data management and system organization. Together, these algorithms reduce waiting time, improve emergency response, and optimize hospital workflow.

Results

Overview

The Smart Hospital Queue Management System was successfully developed and tested as a web-based application. The system effectively manages patient registration, appointment scheduling, priority-based queue management, doctor allocation, and real-time queue tracking. The implementation of Data Structures and Algorithms significantly improves the efficiency of patient scheduling compared to the traditional First-Come-First-Served (FCFS) approach.

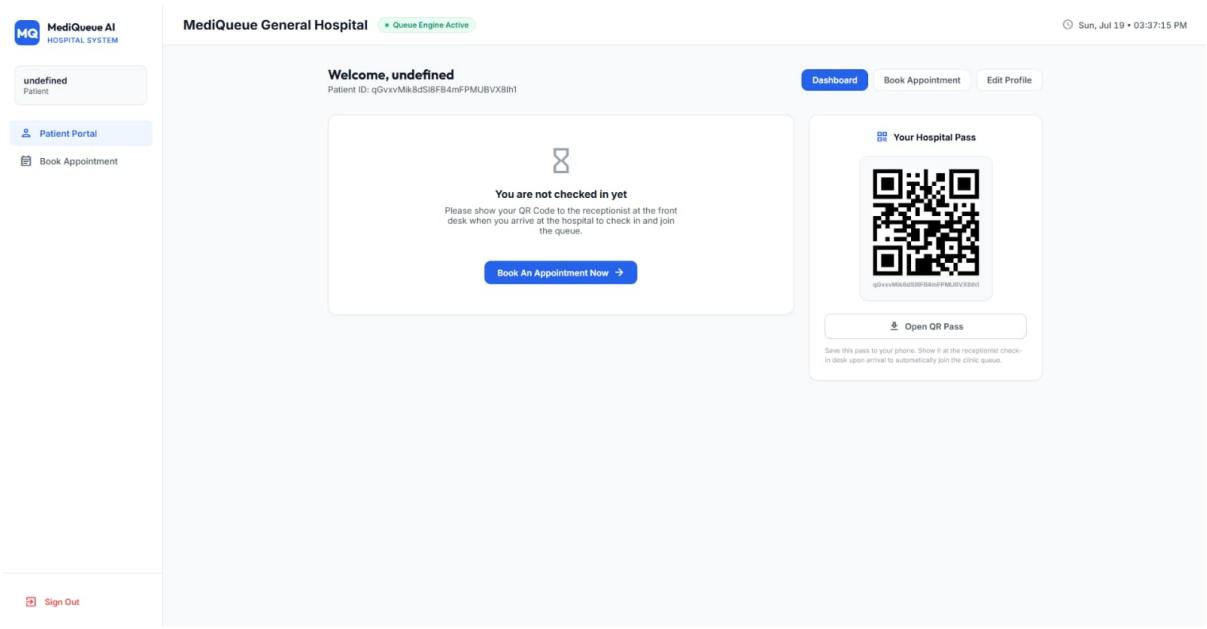
The application provides separate interfaces for patients, doctors, receptionists, and administrators. All modules work together to ensure smooth hospital operations while reducing patient waiting time and improving emergency response.

Home Page

The home page serves as the entry point of the application. It provides users with information about the hospital queue management system and allows them to navigate to the login or registration page.

Result:

- User-friendly interface.
- Easy navigation.
- Responsive design across different screen sizes.

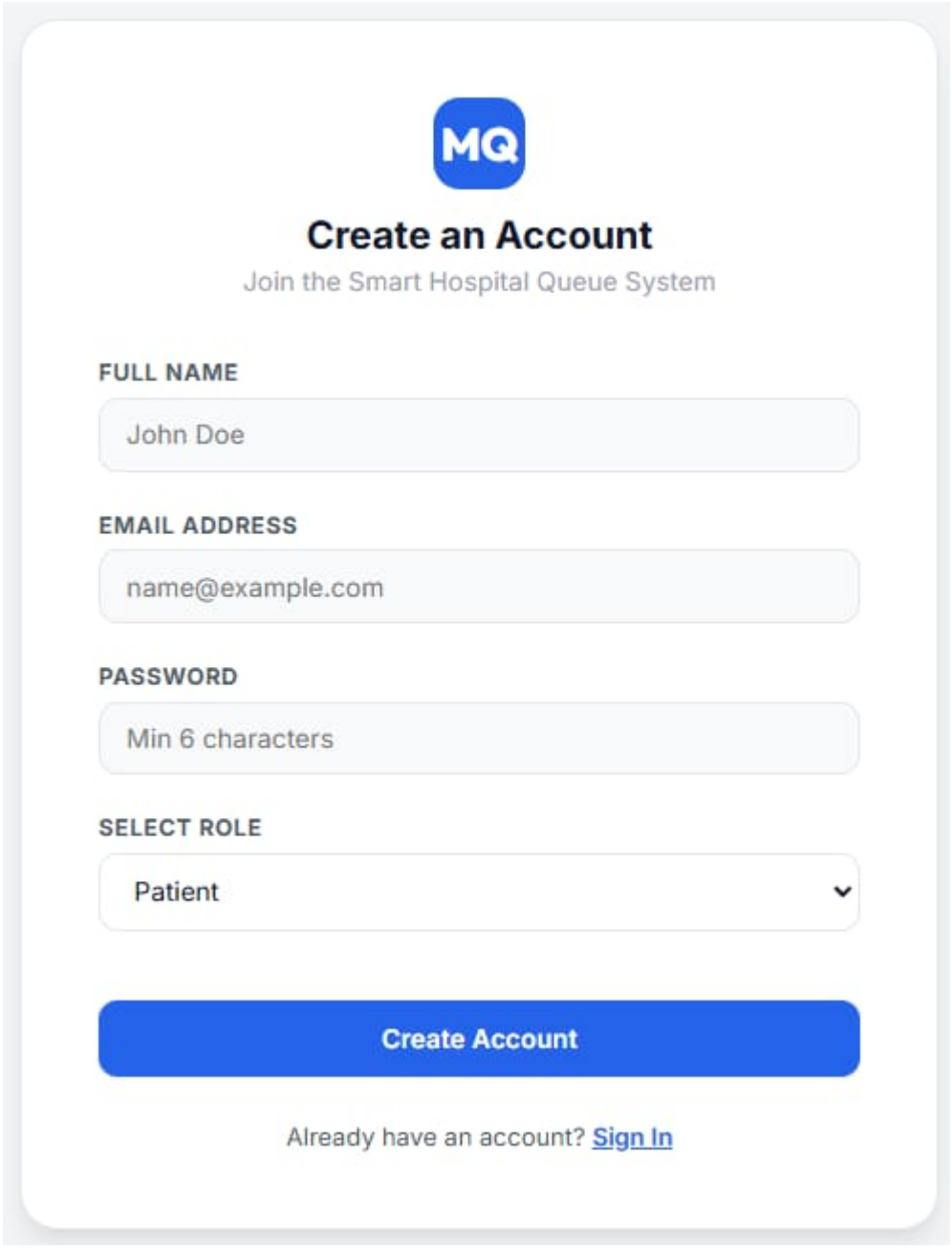


Patient Registration

Patients can register by providing their personal details and medical information. After successful registration, the system generates a patient profile and stores the information securely in the database.

Result:

- Successful patient registration.
- Secure storage of patient details.
- Automatic profile creation.

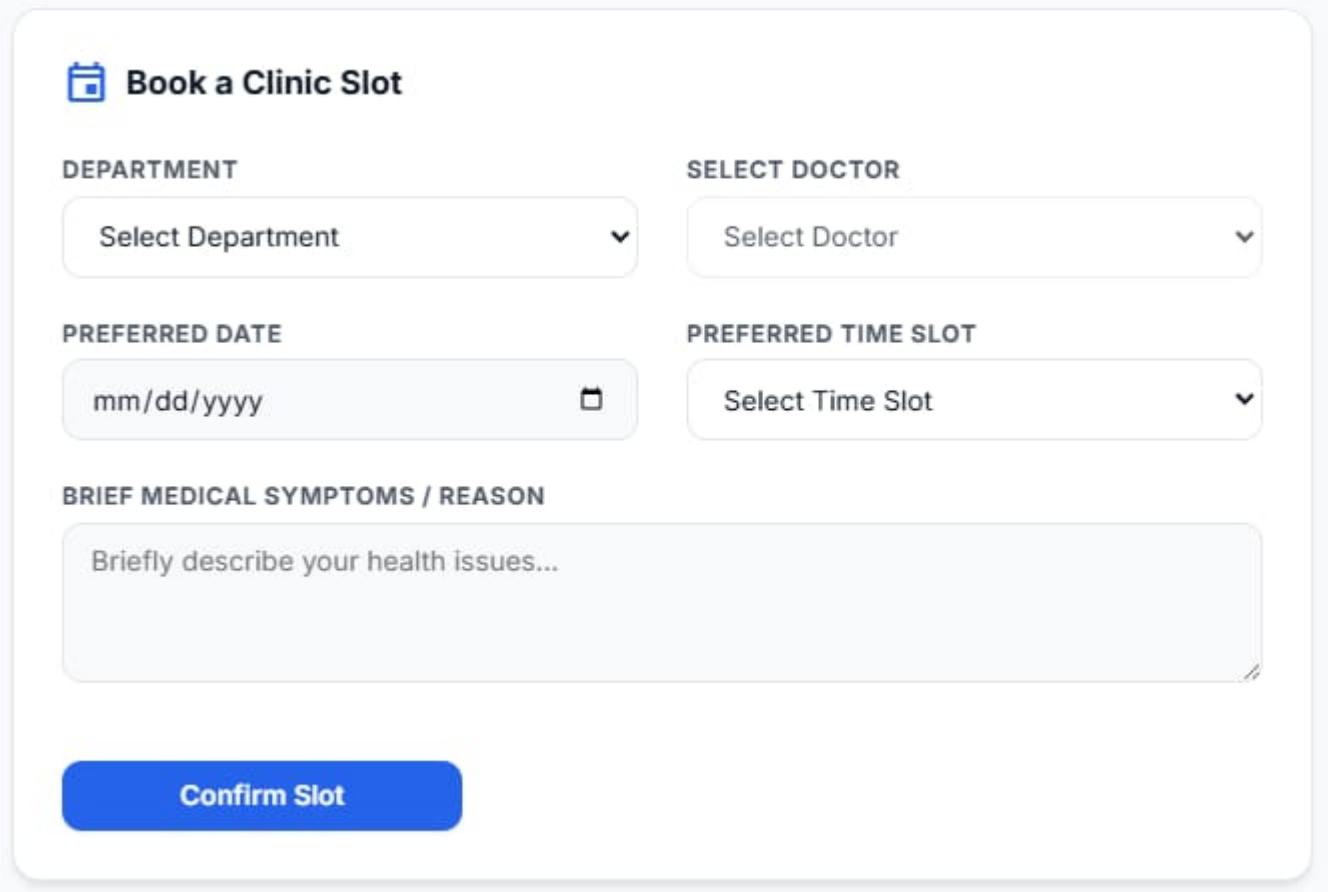

MQ
Create an Account
Join the Smart Hospital Queue System
FULL NAME
John Doe
EMAIL ADDRESS
name@example.com
PASSWORD
Min 6 characters
SELECT ROLE
Patient
Create Account
Already have an account? Sign In

Appointment Booking and Priority Assignment

Patients can book appointments by selecting the required department and doctor. Based on emergency level, age, waiting time, and appointment status, the system calculates a priority score and places the patient in the Priority Queue.

Result:

- Automatic priority score calculation.
- Efficient appointment scheduling.
- Fair patient prioritization.

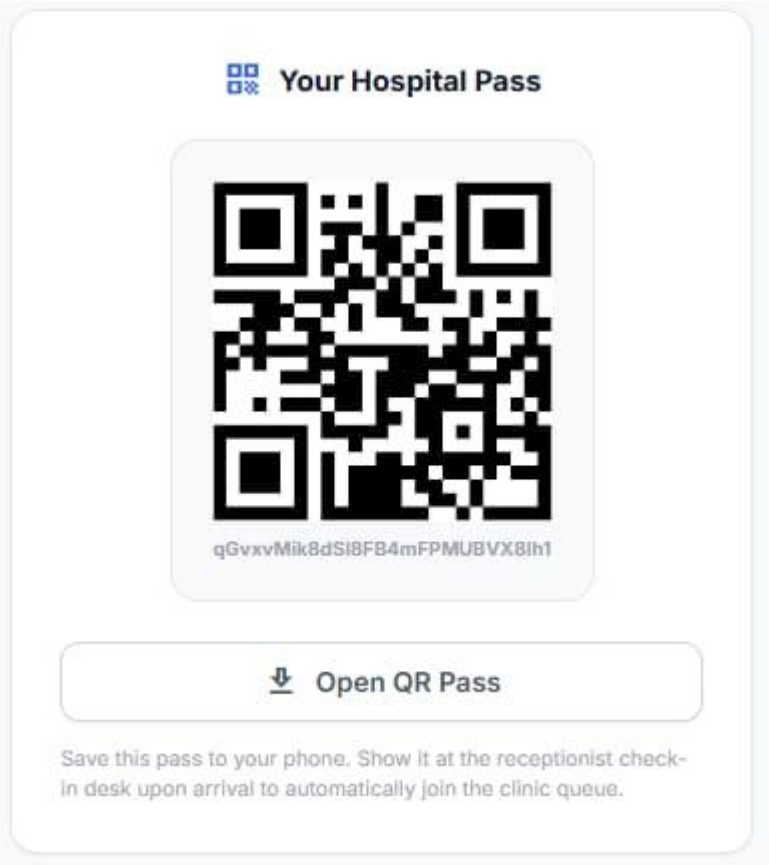


Priority Queue Management

The Priority Queue is the core functionality of the system. Instead of following the First-Come-First-Served approach, patients are arranged according to their calculated priority score using a Binary Max Heap.

Result:

- Emergency patients are served first.
- Reduced waiting time.
- Efficient queue management.

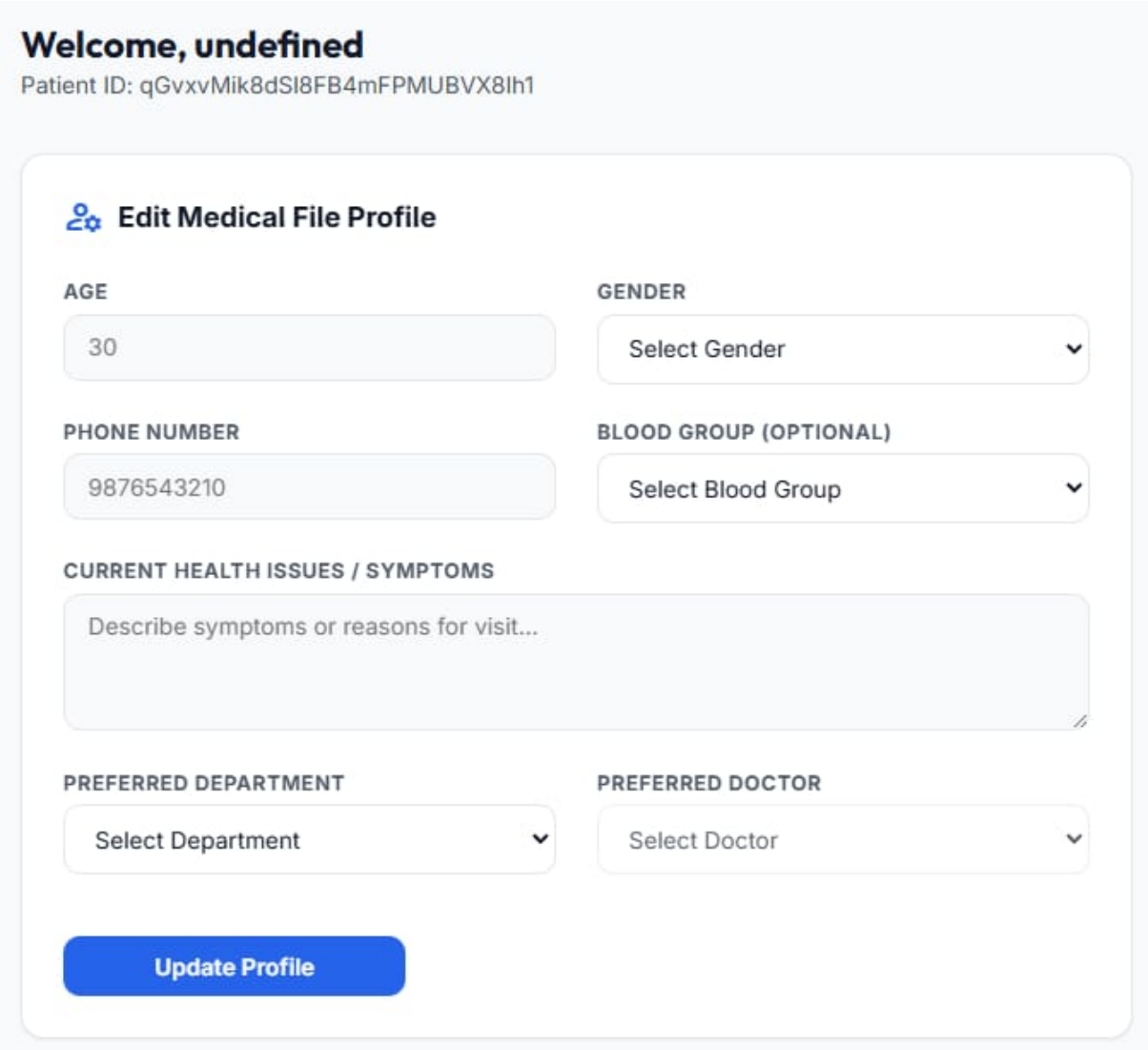


Doctor Dashboard

Doctors can log into the system and view the list of assigned patients. The dashboard displays the highest-priority patient first, allowing doctors to provide timely treatment to critical cases.

Result:

- Automatic patient assignment.
- Real-time queue updates.
- Improved consultation workflow.



Performance Evaluation

The implementation of efficient Data Structures improves overall system performance.

Operation	Data Structure Used	Time Complexity
Insert Patient	Priority Queue (Heap)	O(log n)
Remove Highest Priority Patient	Priority Queue (Heap)	O(log n)
Search Patient	HashMap	O(1)
Queue Operations	Queue	O(1)
Graph Traversal	Graph	O(V + E)

The observed results show that the proposed system performs queue operations efficiently while maintaining quick access to patient records and hospital data.

Overall Results

The Smart Hospital Queue Management System successfully fulfills its objectives by implementing a priority-based patient scheduling mechanism. The system minimizes patient waiting time, improves emergency case handling, and enhances hospital workflow through efficient use of Data Structures and Algorithms. The integration of Priority Queue, Binary Max Heap, Queue, HashMap, and Graph demonstrates the practical application of DSA concepts in solving real-world healthcare management problems.

6. Conclusion

The **Smart Hospital Queue Management System (MediQueue AI)** successfully demonstrates how **Data Structures and Algorithms (DSA)** can be applied to solve real-world healthcare management problems. The project replaces the traditional First-Come-First-Served (FCFS) approach with a **priority-based queue management system**, ensuring that patients with critical medical conditions receive timely treatment while maintaining fairness for all patients.

The implementation of **Priority Queue** using a **Binary Max Heap** serves as the core of the system, enabling efficient patient prioritization based on factors such as emergency level, age, waiting time, and appointment status. Additional data structures, including **Queue**, **HashMap**, and **Graph**, contribute to efficient patient management, fast record retrieval, and logical organization of hospital resources. The use of these DSA concepts improves system performance while reducing computational complexity for common operations.

The developed application provides dedicated interfaces for patients, doctors, receptionists, and administrators, allowing seamless patient registration, appointment scheduling, real-time queue monitoring, doctor assignment, and administrative management. The integration of modern web technologies such as **React.js**, **TypeScript**, **Firebase Authentication**, and **Firestore Database** ensures that the application is responsive, secure, scalable, and easy to maintain.

Overall, the project successfully achieves its primary objectives of reducing patient waiting time, improving emergency response, optimizing hospital workflow, and demonstrating the practical implementation of advanced Data Structures and Algorithms in a real-world application. The system can be further enhanced by incorporating AI-based disease prediction, QR code check-in, SMS and email notifications, online consultation, mobile application support, and integration with Electronic Health Record (EHR) systems. These future enhancements would further increase the efficiency, accessibility, and scalability of the proposed solution.

References

[1] React Documentation, "React – A JavaScript Library for Building User Interfaces." Available: https://react.dev/. Accessed: July 2026.

[2] Firebase Documentation, "Firebase Authentication and Cloud Firestore." Available: https://firebase.google.com/docs. Accessed: July 2026.

[3] Tailwind CSS Documentation, "Tailwind CSS Framework." Available: https://tailwindcss.com/docs. Accessed: July 2026.

[4] Vite Documentation, "Next Generation Frontend Tooling." Available: https://vitejs.dev/. Accessed: July 2026.

[5] Mozilla Developer Network (MDN), "JavaScript Documentation." Available: https://developer.mozilla.org/. Accessed: July 2026.

[6] GitHub Repository, "MediQueue AI – Smart Hospital Queue Management System." Available: https://github.com/Amrittr/MediQueue-AI. Accessed: July 2026.

Source Code:

MediQueue-AI-main.zip

Deployment Link:

https://medi-queue-ai-beige.vercel.app/

GitHub Link:

https://github.com/Amrittr/MediQueue-AI